3 HAFTA

C# ile Nesneye Yönelik Programlama · Ders Notu

OOP'nin Dört Temel Prensibi

OOP'nin Omurgası

Bu haftanın kazanımları

1. **Encapsulation** uygulayarak veri bütünlüğünü koruyan sınıflar tasarlayabilir.
2. **Inheritance** ile sınıf hiyerarşisi kurabilir; **base** anahtar kelimesini kullanabilir.
3. **Overloading** ile **overriding** arasındaki farkı açıklayabilir ve ikisini de uygulayabilir.
4. **virtual**, **override**, **new** ve **sealed** anahtar kelimelerini doğru yerde kullanabilir.
5. **Abstract class** ve **abstract method** tanımlayarak ortak sözleşmeler oluşturabilir.

İçindekiler

1 **Dört Prensibe Genel Bakış**
Prensipler neyi çözer? · Aralarındaki ilişki · Ortak örnek senaryo

2 **Encapsulation — Kapsülleme**
Bilgi gizleme · private field + public property · Doğrulama · readonly ve init · field keyword (C# 14)

3 **Inheritance — Kalıtım**
Base ve derived sınıflar · base anahtar kelimesi · protected üyeler · Object sınıfı · sealed · is / as

4 **Polymorphism — Çok Biçimlilik**
Compile-time ve runtime · Overloading · Overriding · virtual, override, new · Upcasting / downcasting

5 **Abstraction — Soyutlama**
abstract class · abstract method · Soyut ve somut üyeler · Abstract vs interface · Interface'e köprü

6 **Dört Prensip Bir Arada — Uygulama**
Ödeme sistemi örneği · Sınıf hiyerarşisi tasarımı · Sık yapılan hatalar

★ **Haftanın Özeti, Sözlük, Alıştırmalar ve Ekstra Okuma**
Özet · Terimler sözlüğü · Alıştırmalar · Kaynak bağlantıları

1. Dört Prensibe Genel Bakış

Bu bölümün anahtar kelimeleri

pillar (temel direk) · **coupling** (bağımlılık) · **cohesion** (uyum / bütünlük) · **contract** (sözleşme) · **hierarchy** (hiyerarşi)

Geçen hafta sınıf, nesne, property ve constructor gibi **yapı taşlarını** öğrendik. Bu hafta bu taşları sağlam bir mimariye dönüştüren **dört prensibi** işleyeceğiz. Bu prensipler bir sözdizimi kuralı değil, bir **tasarım disiplinidir**: dili bilmek yetmez, ne zaman kullanılacağını bilmek gerekir.

Tablo 1.1 — Her prensip hangi problemi çözer?

Prensip	Çözdüğü problem	Anahtar araçları
Encapsulation (Kapsülleme)	Nesnenin verisi dışarıdan bozulabiliyor; geçersiz duruma düşebiliyor.	`private` field, property, `readonly`, `init`
Inheritance (Kalıtım)	Benzer sınıflarda aynı kod tekrar tekrar yazılıyor.	`:` operatörü, `base`, `protected`
Polymorphism (Çok biçimlilik)	Her alt tip için ayrı `if` yazmak gerekiyor; yeni tip ekleyince eski kod değişiyor.	`virtual`, `override`, üst tip referansı
Abstraction (Soyutlama)	Ortak davranış zorunlu kılınamıyor; eksik uygulama çalışma zamanında patlıyor.	`abstract` class ve method, `interface`

> **[i] Prensipler birbirini besler**
>
> **Encapsulation** nesneyi güvenli hâle getirir. **Inheritance** ortak kodu yukarı taşır. **Abstraction** alt sınıfların uyması gereken sözleşmeyi tanımlar. **Polymorphism** ise bu sözleşme sayesinde tek bir kod satırının farklı tiplerle çalışmasını sağlar. Dördü birlikte çalıştığında ortaya **değişime açık, bozulmaya kapalı** bir tasarım çıkar.

1.1 Bu hafta boyunca kullanacağımız örnek

Konuları bağlantılı tutmak için bir **personel yönetim sistemi** üzerinden ilerleyeceğiz. Hedef sınıf hiyerarşimiz şudur:

Kod 1.1 — Hedeflediğimiz yapı

```
                Personel  (abstract)
                    |
        +-----------+-------------+
        |           |             |
   Muhendis      Yonetici       Stajyer

  Ortak: Ad, SicilNo, IseBaslama, TemelMaas
  Farkli: her rol icin MaasHesapla() farkli calisir
```

2. Encapsulation — Kapsülleme

Bu bölümün anahtar kelimeleri

encapsulation (kapsülleme) · **information hiding** (bilgi gizleme) · **invariant** (değişmez kural) · **validation** (doğrulama) · **getter / setter** (okuyucu / yazıcı) · **immutable** (değiştirilemez) · **backing field** (arka plan alanı)

2.1 Problem: korumasız veri

Encapsulation, nesnenin iç verisini dış dünyadan gizleyip ona yalnızca kontrollü kanallardan erişilmesini sağlamaktır. Amaç, nesnenin her an **geçerli bir durumda** kalmasını garanti etmektir. Bu garantilere **invariant (değişmez kural)** denir.

Kod 2.1 — Kapsüllenmemiş sınıf: her şey serbest

```
class BankaHesabiKotu
{
    public decimal Bakiye;        // herkes istedigini yazabilir
    public string Iban;
}

var h = new BankaHesabiKotu();
h.Bakiye = -50_000;               // gecersiz durum, kimse engellemedi
h.Iban = "merhaba";               // gecersiz IBAN
```

[x] Buradaki hata sözdizimsel değil, tasarımsaldır

Kod sorunsuz derlenir ve çalışır. Ancak sınıf, **kendi kurallarını koruyamaz** hâle gelmiştir. Bakiyenin negatif olamayacağı bilgisi artık sınıfın içinde değil, onu kullanan **her yerde** tekrar tekrar yazılması gereken bir yüke dönüşmüştür.

2.2 Çözüm: private field + public property

Kod 2.2 — Kapsüllenmiş sınıf

```
public class BankaHesabi
{
    private decimal _bakiye;                  // disariya KAPALI
    private readonly List<string> _hareketler = new();

    public string Iban { get; }               // sadece ctor'da atanir
    public string SahipAdi { get; private set; }

    // Hesaplanan, salt okunur property
    public decimal Bakiye => _bakiye;
    public IReadOnlyList<string> Hareketler => _hareketler;

    public BankaHesabi(string iban, string sahipAdi, decimal acilisBakiyesi = 0)
    {
        if (string.IsNullOrWhiteSpace(iban) || iban.Length != 26)
            throw new ArgumentException("Gecersiz IBAN.", nameof(iban));
        if (acilisBakiyesi < 0)
            throw new ArgumentOutOfRangeException(nameof(acilisBakiyesi));

        Iban = iban;
        SahipAdi = sahipAdi;
```

```
        _bakiye = acilisBakiyesi;
    }

    public void ParaYatir(decimal tutar)
    {
        if (tutar <= 0)
            throw new ArgumentOutOfRangeException(nameof(tutar), "Tutar pozitif olmali.");

        _bakiye += tutar;
        _hareketler.Add($"{DateTime.Now:d} +{tutar:C}");
    }

    public bool ParaCek(decimal tutar)
    {
        if (tutar <= 0 || tutar > _bakiye)
            return false;                   // invariant korunur

        _bakiye -= tutar;
        _hareketler.Add($"{DateTime.Now:d} -{tutar:C}");
        return true;
    }
}
```

Artık `_bakiye` yalnızca `ParaYatir` ve `ParaCek` üzerinden değişebilir. Hesabın negatife düşmesi **mümkün değildir** — bu kural sınıfın içine gömülmüştür.

2.3 Doğrulamayı property'nin içine koymak

Kod 2.3 — Klasik yöntem: backing field ile

```
public class Urun
{
    private decimal _fiyat;

    public decimal Fiyat
    {
        get => _fiyat;
        set
        {
            if (value < 0)
                throw new ArgumentOutOfRangeException(nameof(value), "Fiyat negatif olamaz.");
            _fiyat = Math.Round(value, 2);   // normalizasyon
        }
    }
}
```

Kod 2.4 — C# 14 yöntemi: field anahtar kelimesi

```
public class Urun
{
    // Backing field'i derleyici olusturur; 'field' ile ona erisiriz.
    public decimal Fiyat
    {
        get;
        set
        {
            if (value < 0)
                throw new ArgumentOutOfRangeException(nameof(value));
            field = Math.Round(value, 2);    // _fiyat yazmaya gerek yok
        }
    }
}
```

> **[+] C# 14 ile gelen field anahtar kelimesi**
>
> Yıllardır her doğrulamalı property için elle bir `_fiyat` alanı yazmak gerekiyordu. C# 14'te `field` anahtar kelimesi bu alanı sizin yerinize üretir ve kod tekrarını ortadan kaldırır. Alanın adını yazmadığınız için yanlışlıkla doğrudan alana erişme riski de kalkar.

2.4 Değiştirilemez (immutable) nesneler

Bazı nesnelerin oluşturulduktan sonra hiç değişmemesi istenir (para birimi, tarih aralığı, konfigürasyon). Bu, kapsüllemenin en güçlü biçimidir: değişmeyen bir nesne **hiçbir zaman geçersiz duruma düşemez**.

Kod 2.5 — Üç immutability yöntemi

```
// 1) readonly field + sadece get property
public class ParaBirimi
{
    private readonly string _kod;
    public string Kod => _kod;
    public ParaBirimi(string kod) => _kod = kod;
}

// 2) init-only property (C# 9+)
public class Konfigurasyon
{
    public required string SunucuAdi { get; init; }
    public int Port { get; init; } = 5432;
}

// 3) record: immutable veri tasiyicisi (en kisa yol)
public record Nokta(int X, int Y);

var k = new Konfigurasyon { SunucuAdi = "db01" };
// k.Port = 1234;                  // HATA: init-only

var n1 = new Nokta(3, 4);
var n2 = n1 with { Y = 9 };        // kopya olusturur, orijinali bozmaz
Console.WriteLine(n1);             // Nokta { X = 3, Y = 4 }
Console.WriteLine(n2);             // Nokta { X = 3, Y = 9 }
```

2.5 Encapsulation kontrol listesi

Yapılmalı	Yapılmamalı
Field'ları `private` tanımlayın	`public` field yazmayın
Dışa açılan her veriyi property yapın	Property'yi boş bir geçiş noktası olarak bırakıp doğrulamayı çağıran koda bırakmayın
Doğrulamayı constructor ve setter içine koyun	Aynı doğrulamayı kullanan her yerde tekrarlamayın
Koleksiyonları `IReadOnlyList<T>` olarak açın	İç listenin kendisini `public` döndürmeyin
Değişmesi gerekmeyen veriyi `init` / `readonly` yapın	Her property'ye refleks olarak `set` eklemeyin

3. Inheritance — Kalıtım

Bu bölümün anahtar kelimeleri

`inheritance` (kalıtım) · **`base class`** (temel sınıf) · **`derived class`** (türetilmiş sınıf) · **`superclass / subclass`** (üst sınıf / alt sınıf) · **`is-a relationship`** (-dır ilişkisi) · **`composition`** (bileşim) · **`sealed`** (mühürlü)

3.1 Kalıtım nedir?

Inheritance, bir sınıfın başka bir sınıfın üyelerini **devralmasıdır**. Devreden sınıfa **base class (temel sınıf)**, devralana **derived class (türetilmiş sınıf)** denir. Amaç kod tekrarını ortadan kaldırmak ve gerçek dünyadaki **"-dır" ilişkisini** modellemektir: "Bir mühendis, bir personeldir."

Kod 3.1 — Temel ve türetilmiş sınıflar

```
public class Personel
{
    public string Ad { get; set; }
    public string SicilNo { get; set; }
    protected decimal TemelMaas { get; set; }   // alt siniflar gorebilir

    public Personel(string ad, string sicilNo, decimal temelMaas)
    {
        Ad = ad;
        SicilNo = sicilNo;
        TemelMaas = temelMaas;
    }

    public void GirisYap() => Console.WriteLine($"{Ad} giris yapti.");

    public virtual decimal MaasHesapla() => TemelMaas;
}

// ':' = "bu sinif Personel'den turer"
public class Muhendis : Personel
{
    public string Uzmanlik { get; set; }

    // base(...) ile temel sinifin constructor'i cagrilir - ZORUNLU
    public Muhendis(string ad, string sicilNo, decimal temelMaas, string uzmanlik)
        : base(ad, sicilNo, temelMaas)
    {
        Uzmanlik = uzmanlik;
    }

    public override decimal MaasHesapla() => TemelMaas * 1.20m;   // %20 teknik zam
}

public class Yonetici : Personel
{
    public int EkipBuyuklugu { get; set; }

    public Yonetici(string ad, string sicilNo, decimal temelMaas, int ekip)
        : base(ad, sicilNo, temelMaas)
    {
        EkipBuyuklugu = ekip;
    }
```

```
    public override decimal MaasHesapla() => TemelMaas * 1.50m + EkipBuyuklugu * 1_000m;
}
```

Kod 3.2 — Kullanımı

```
var m = new Muhendis("Deniz", "M-101", 50_000m, "Gomulu Sistemler");
var y = new Yonetici("Selin", "Y-007", 60_000m, 8);

m.GirisYap();                          // Personel'den devralindi
Console.WriteLine(m.MaasHesapla());    // 60000  -> Muhendis versiyonu
Console.WriteLine(y.MaasHesapla());    // 98000  -> Yonetici versiyonu
```

3.2 base anahtar kelimesi

`base`, temel sınıfa erişmek için kullanılır. İki yerde karşınıza çıkar:

```
Kod 3.ublic class Stajyer : Personel
{
    public int SureAy { get; set; }

    // 1) Temel sinifin constructor'ini cagirmak
    public Stajyer(string ad, string sicil, decimal maas, int sureAy)
        : base(ad, sicil, maas)
    {
        SureAy = sureAy;
    }
    // 2) Temel sinifin metodunu genisletmek (tamamen degistirmek degil)
    public override decimal MaasHesapla()
    {
        decimal taban = base.MaasHesapla();    // once ustteki hesabi al
        return taban * 0.40m;                  // sonra stajyer katsayisini uygula
    }
}
```

> **[!] Constructor'lar kalıtımla geçmez**
> Türetilmiş sınıf, temel sınıfın constructor'larını **devralmaz**. Temel sınıfın parametresiz bir constructor'ı yoksa, türetilmiş sınıfın her constructor'ı `: base(...)` ile hangi temel constructor'ı çağıracağını **açıkça belirtmek zorundadır**.

3.3 Her sınıfın atası: Object

C#'ta açıkça bir temel sınıf belirtmeseniz bile her sınıf otomatik olarak `System.Object`'ten türer. Bu yüzden her nesnede şu metotlar bulunur:

Kod 3.4 — ToString() ezmek (override)

```
public class Personel
{
    public string Ad { get; set; }
    public string SicilNo { get; set; }

    // Object.ToString() varsayilan olarak tip adini dondurur: "Personel"
    public override string ToString() => $"{SicilNo} - {Ad}";
}

var p = new Personel { Ad = "Deniz", SicilNo = "M-101" };
Console.WriteLine(p);            // M-101 - Deniz   (WriteLine ToString'i cagirir)
```

Tablo 3.1 — Object sınıfının ezilebilen metotları

Metot	Varsayılan davranış	Ne zaman ezilir?
`ToString()`	Tipin tam adını döndürür	Nesnenin okunabilir özetini vermek için
`Equals(object)`	Referans eşitliği (aynı nesne mi?)	İçerik eşitliği tanımlamak için
`GetHashCode()`	Nesneye özel sayı	`Equals` ezildiyse **mutlaka** birlikte ezilir
`GetType()`	Çalışma zamanındaki gerçek tip	Ezilemez (`sealed`)

3.4 sealed — kalıtımı durdurmak

Kod 3.5

```
public sealed class VergiHesaplayici        // bu siniftan TUREYEMEZSINIZ
{
    public decimal Hesapla(decimal brut) => brut * 0.15m;
}

// public class Ozel : VergiHesaplayici { }   // DERLEME HATASI

public class Yonetici : Personel
{
    // Bu metot artik Yonetici'den turetilenler tarafindan EZILEMEZ
    public sealed override decimal MaasHesapla() => TemelMaas * 1.50m;
}
```

3.5 is ve as operatörleri

Kod 3.6 — Tip kontrolü ve güvenli dönüşüm

```
Personel p = new Muhendis("Deniz", "M-101", 50_000m, "Yazilim");

// is: tip kontrolu + pattern matching ile tek adimda donusturme
if (p is Muhendis m)
    Console.WriteLine(m.Uzmanlik);        // Yazilim

// as: donusturemezse EXCEPTION FIRLATMAZ, null dondurur
Yonetici? y = p as Yonetici;
Console.WriteLine(y is null ? "Yonetici degil" : y.EkipBuyuklugu.ToString());

// dogrudan cast: donusturemezse InvalidCastException FIRLATIR
// Yonetici z = (Yonetici)p;              // CALISMA ZAMANI HATASI

// switch ile pattern matching - en okunakli yontem
string rol = p switch
{
    Yonetici => "Yonetici",
    Muhendis => "Muhendis",
    Stajyer  => "Stajyer",
    _        => "Personel"
};
Console.WriteLine(rol);
```

3.6 Ne zaman kalıtım kullanmamalı?

> **[!] "Composition over inheritance" (bileşim, kalıtıma yeğdir)**
>
> Kalıtım güçlü ama **sıkı bağlı** bir ilişkidir: temel sınıftaki her değişiklik tüm alt sınıfları etkiler. Kalıtımı yalnızca gerçek bir **"-dır" (is-a)** ilişkisi varsa kullanın. İlişki **"-sahiptir" (has-a)** ise, o sınıfı miras almak yerine **üye olarak tutun**.

Kod 3.7

```
// YANLIS: bir araba, motor DEGILDIR
// class Araba : Motor { }

// DOGRU: bir arabanin motoru VARDIR -> composition
class Araba
{
    private readonly Motor _motor;         // has-a
    public Araba(Motor motor) => _motor = motor;
    public void Calistir() => _motor.Ates();
}
```

> **[i] C# çoklu kalıtımı desteklemez**
>
> Bir sınıf yalnızca **tek bir** temel sınıftan türeyebilir. Ancak istediğiniz kadar `interface` uygulayabilir. Bu tasarım tercihi, birden çok temel sınıfın aynı üyeyi tanımlaması durumunda ortaya çıkan belirsizliği ("elmas problemi") engeller.

4. Polymorphism — Çok Biçimlilik

Bu bölümün anahtar kelimeleri

`polymorphism` (çok biçimlilik) · **`overloading`** (aşırı yükleme) · **`overriding`** (ezme / geçersiz kılma) · **`virtual`** (sanal) · **`upcasting`** (yukarı dönüşüm) · **`downcasting`** (aşağı dönüşüm) · **`method hiding`** (metot gizleme) · **`late binding`** (geç bağlama)

4.1 İki tür çok biçimlilik

Polymorphism, kelime anlamıyla "çok biçimlilik"tir: **aynı çağrının**, bağlama göre **farklı davranması**. C#'ta iki biçimi vardır ve karıştırılmaları çok yaygındır.

Tablo 4.1 — Overloading ve overriding karşılaştırması

	Overloading (aşırı yükleme)	Overriding (ezme)
Türkçesi	Aşırı yükleme	Geçersiz kılma / ezme
Ne zaman çözülür?	**Compile time** (derleme zamanı)	**Run time** (çalışma zamanı)
Diğer adı	Static / early binding	Dynamic / late binding
Nerede olur?	**Aynı** sınıf içinde	**Temel ve türetilmiş** sınıf arasında
İmza	**Farklı** olmak zorunda	**Aynı** olmak zorunda
Anahtar kelime	(yok)	`virtual` + `override`
Kalıtım gerekir mi?	Hayır	Evet

4.2 Overloading — derleme zamanı çok biçimliliği

Kod 4.1

```
public class Rapor
{
    public void Yazdir(string metin)
        => Console.WriteLine($"Metin: {metin}");

    public void Yazdir(string metin, int kopyaSayisi)
        => Console.WriteLine($"Metin: {metin} x{kopyaSayisi}");

    public void Yazdir(decimal tutar)
        => Console.WriteLine($"Tutar: {tutar:C}");

    public void Yazdir(List<string> satirlar)
        => Console.WriteLine($"{satirlar.Count} satir yazdirildi.");
}

var r = new Rapor();
r.Yazdir("Aylik rapor");          // 1. metot
r.Yazdir("Aylik rapor", 3);       // 2. metot
r.Yazdir(1500.50m);               // 3. metot
r.Yazdir(["a", "b", "c"]);        // 4. metot
```

Derleyici, argümanların tiplerine ve sayısına bakarak hangi metodun çağrılacağına **programı çalıştırmadan önce** karar verir. Bu yüzden buna **static binding (erken bağlama)** denir.

4.3 Overriding — çalışma zamanı çok biçimliliği

Asıl güçlü olan budur. Temel sınıftaki metot `virtual` işaretlenir; türetilmiş sınıf onu `override` ile yeniden yazar. Çağrı yapıldığında, değişkenin **tanımlı tipi** değil, nesnenin **gerçek tipi** belirleyicidir.

Kod 4.2 — Şekil hiyerarşisi

```
public class Sekil
{
    public string Ad { get; set; } = "Sekil";

    public virtual double AlanHesapla() => 0;      // virtual: ezilebilir

    public virtual void Bilgi()
        => Console.WriteLine($"{Ad}: alan = {AlanHesapla():F2}");
}

public class Dikdortgen : Sekil
{
    public double Genislik { get; set; }
    public double Yukseklik { get; set; }

    public Dikdortgen() => Ad = "Dikdortgen";

    public override double AlanHesapla() => Genislik * Yukseklik;
}

public class Daire : Sekil
{
    public double Yaricap { get; set; }

    public Daire() => Ad = "Daire";

    public override double AlanHesapla() => Math.PI * Yaricap * Yaricap;
}

public class Ucgen : Sekil
{
    public double Taban { get; set; }
    public double Yukseklik { get; set; }

    public Ucgen() => Ad = "Ucgen";

    public override double AlanHesapla() => Taban * Yukseklik / 2;
}
```

```
// Hepsi Sekil olarak tutuluyor - UPCASTING (yukari donusum, otomatik)
List<Sekil> sekiller =
[
    new Dikdortgen { Genislik = 4, Yukseklik = 5 },
    new Daire      { Yaricap = 3 },
    new Ucgen      { Taban = 6, Yukseklik = 4 }
];

// TEK bir dongü, UC farkli davranis. Hicbir if / switch yok!
double toplam = 0;
foreach (Sekil s in sekiller)
{
    s.Bilgi();                    // her nesne KENDI AlanHesapla'sini calistirir
    toplam += s.AlanHesapla();
}

Console.WriteLine($"Toplam alan: {toplam:F2}");
```

```
Dikdortgen: alan = 20,00
Daire: alan = 28,27
Ucgen: alan = 12,00
Toplam alan: 60,27
```

> **[+] Polymorphism'in gerçek değeri: değişime kapalılık**
>
> Yarın `Kare` sınıfı eklediğinizde **yukarıdaki döngüyü hiç değiştirmezsiniz**. `if (sekil is Daire) ... else if (sekil is Ucgen) ...` şeklinde yazsaydınız, her yeni tipte bu zinciri ve onun gibi onlarca yeri güncellemeniz gerekirdi. Buna **Open/Closed Principle** (genişlemeye açık, değişikliğe kapalı) denir.

4.4 new — method hiding (metot gizleme)

`new` anahtar kelimesi, temel sınıftaki metodu ezmez; onu **gizler**. Sonuç, `override`'dan tamamen farklıdır ve genellikle **istenmeyen** bir durumdur.

```
class Temel
{
    public virtual void Selam() => Console.WriteLine("Temel.Selam");
    public void Merhaba()       => Console.WriteLine("Temel.Merhaba");
}

class Turetilmis : Temel
{
    public override void Selam()   => Console.WriteLine("Turetilmis.Selam");
    public new void Merhaba()      => Console.WriteLine("Turetilmis.Merhaba");
}

Temel t = new Turetilmis();     // degisken tipi Temel, nesne tipi Turetilmis

t.Selam();       // Turetilmis.Selam   -> OVERRIDE: nesnenin gercek tipi kazanir
t.Merhaba();     // Temel.Merhaba      -> NEW: degiskenin tipi kazanir (!)

Turetilmis d = new Turetilmis();
d.Merhaba();     // Turetilmis.Merhaba
```

> **[x] new'i kaza eseri kullanmayın**
>
> Temel sınıfta `virtual` olmayan bir metodu türetilmiş sınıfta aynı adla yazarsanız derleyici uyarı verir ve örtük olarak `new` davranışı uygulanır. Bu, bulunması çok zor hatalara yol açar. Niyetiniz ezmekse **temel metodu `virtual` yapın** ve `override` kullanın.

4.5 Upcasting ve downcasting

Kod 4.5

```
Daire d = new Daire { Yaricap = 3 };

// UPCASTING: alt tipten ust tipe - her zaman guvenli, otomatik
Sekil s = d;
Console.WriteLine(s.AlanHesapla());   // 28.27 -> yine Daire'nin metodu

// s.Yaricap;   // HATA: Sekil tipinde Yaricap yok (derleyici gormez)

// DOWNCASTING: ust tipten alt tipe - RISKLI, acikca yazilmali
if (s is Daire d2)                    // guvenli yol
    Console.WriteLine(d2.Yaricap);    // 3

Sekil baska = new Ucgen();
// Daire hata = (Daire)baska;         // InvalidCastException!
Daire? guvenli = baska as Daire;      // null doner, patlamaz
Console.WriteLine(guvenli is null);   // True
```

5. Abstraction — Soyutlama

Bu bölümün anahtar kelimeleri

`abstraction` (soyutlama) · `abstract class` (soyut sınıf) · `abstract method` (soyut metot) · `concrete class` (somut sınıf) · `interface` (arayüz) · `contract` (sözleşme) · `implementation` (gerçekleştirim)

5.1 Soyutlama nedir?

Abstraction, bir kavramın **ne yaptığını** tanımlayıp **nasıl yaptığını** gizlemektir. Arabayı sürerken direksiyonu çevirirsiniz; direksiyon kutusunun içinde ne olduğunu bilmeniz gerekmez. Direksiyon, arabanın **soyutlamasıdır**.

Kod tarafında bu şu anlama gelir: bir üst sınıf, alt sınıfların **uymak zorunda olduğu bir sözleşme** tanımlar ama o sözleşmeyi kendisi doldurmaz.

5.2 abstract class ve abstract method

Önceki bölümdeki `Sekil` sınıfında bir sorun vardı: `new Sekil()` yazıp alanı sıfır olan anlamsız bir nesne üretilebiliyordu. Ayrıca yeni bir şekil yazan geliştirici `AlanHesapla`'yı ezmeyi unutabilirdi. `abstract` her ikisini de çözer.

Kod 5.1 — Soyut sınıf

```
public abstract class Sekil
{
    // SOMUT (concrete) uye: govdesi var, tum alt siniflar kullanir
    public string Ad { get; protected set; } = "Sekil";

    // SOYUT (abstract) uye: govdesi YOK, alt sinif YAZMAK ZORUNDA
    public abstract double AlanHesapla();
    public abstract double CevreHesapla();

    // virtual: govdesi var ama istenirse ezilebilir
    public virtual void Bilgi()
        => Console.WriteLine($"{Ad}: alan={AlanHesapla():F2}, cevre={CevreHesapla():F2}");
}

// var s = new Sekil();   // DERLEME HATASI: soyut sinif ornek alinamaz

public class Daire : Sekil
{
    public double Yaricap { get; set; }

    public Daire(double yaricap)
    {
        Yaricap = yaricap;
        Ad = "Daire";
    }

    // abstract uyeler override ile DOLDURULMAK ZORUNDA
    public override double AlanHesapla() => Math.PI * Yaricap * Yaricap;
    public override double CevreHesapla() => 2 * Math.PI * Yaricap;
}

public class Kare : Sekil
{
    public double Kenar { get; set; }
```

```
    public Kare(double kenar)
    {
        Kenar = kenar;
        Ad = "Kare";
    }

    public override double AlanHesapla() => Kenar * Kenar;
    public override double CevreHesapla() => 4 * Kenar;

    public override void Bilgi()                // virtual uyeyi ezmek opsiyonel
        => Console.WriteLine($"Kare (kenar={Kenar}) -> alan={AlanHesapla()}");
}
```

Tablo 5.1 — abstract sınıfın kuralları

Kural	Açıklama
Örneklenemez	`new AbstractSinif()` derleme hatası verir
Hem soyut hem somut üye içerebilir	Ortak kodu paylaşır, farklıyı zorunlu kılar
`abstract` üyenin gövdesi olamaz	Sadece imza yazılır, sonuna `;` konur
Soyut üye ancak soyut sınıfta olur	Somut bir sınıfta `abstract` metot tanımlanamaz
Constructor'ı olabilir	Alt sınıflar `base(...)` ile çağırır
`sealed` ile birleşemez	Soyut sınıfın türetilmesi zaten zorunludur

5.3 Abstract class mı, interface mi?

`interface` konusunu ilerleyen haftalarda ayrıntılı işleyeceğiz; ancak ikisi arasındaki seçim, soyutlamanın kalbindedir.

Tablo 5.2 — Karşılaştırma

	abstract class	interface
İlişki türü	**is-a** (bir tür ... dır)	**can-do** (... yapabilir)
Çoklu kullanım	Yalnızca **bir** tanesinden türetilir	Bir sınıf **birçok** interface uygulayabilir
Field tutabilir mi?	Evet	Hayır (yalnızca property/metot imzası)
Constructor	Olabilir	Olamaz
Erişim belirleyici	Serbest (`protected` dâhil)	Üyeler örtük `public`
Ortak kod paylaşımı	Güçlü — asıl amacı budur	Sınırlı (C# 8+ default implementation)
Tipik örnek	`Sekil`, `Personel`	`IDisposable`, `IComparable`

Kod 5.2 — İkisi birlikte

```
public interface IRaporlanabilir        // can-do sozlesmesi
{
    string RaporSatiri();
}

public abstract class Personel : IRaporlanabilir
{
    public string Ad { get; set; } = "";
    public string SicilNo { get; set; } = "";
    protected decimal TemelMaas { get; set; }

    public abstract decimal MaasHesapla();      // her rol farkli hesaplar

    public virtual string RaporSatiri()
        => $"{SicilNo,-8} {Ad,-15} {MaasHesapla(),12:C}";
}
```

6. Dört Prensip Bir Arada — Uygulama

Şimdi öğrendiğimiz dört prensibi tek bir örnekte birleştirelim: bir **ödeme sistemi**. Sistem farklı ödeme yöntemlerini desteklemeli ve yeni yöntem eklemek mevcut kodu değiştirmeyi gerektirmemelidir.

Kod 6.1 — Tam örnek

```
// ===== ABSTRACTION: ortak sozlesme =====
public abstract class Odeme
{
    // ===== ENCAPSULATION: veri korunuyor =====
    private decimal _tutar;

    public decimal Tutar
    {
        get => _tutar;
        private set
        {
            if (value <= 0)
                throw new ArgumentOutOfRangeException(nameof(value),
                    "Odeme tutari pozitif olmalidir.");
            _tutar = value;
        }
    }

    public string IslemNo { get; }
    public DateTime Tarih { get; } = DateTime.Now;

    protected Odeme(decimal tutar)
    {
        Tutar = tutar;
        IslemNo = Guid.NewGuid().ToString()[..8].ToUpper();
    }

    // Alt siniflarin doldurmak ZORUNDA oldugu uyeler
    public abstract string YontemAdi { get; }
    public abstract decimal KomisyonHesapla();

    // Ortak davranis - tum odemelerde ayni
    public decimal ToplamTutar() => Tutar + KomisyonHesapla();

    public override string ToString()
        => $"[{IslemNo}] {YontemAdi,-12} {Tutar,10:C} + {KomisyonHesapla(),8:C} " +
           $"= {ToplamTutar(),10:C}";
}

// ===== INHERITANCE: ortak kod yukarida, fark asagida =====
public class KrediKartiOdemesi : Odeme
{
    public int TaksitSayisi { get; }

    public KrediKartiOdemesi(decimal tutar, int taksit = 1) : base(tutar)
    {
        if (taksit is < 1 or > 12)
            throw new ArgumentOutOfRangeException(nameof(taksit), "Taksit 1-12 arasi olmali.");
        TaksitSayisi = taksit;
    }

    public override string YontemAdi => "Kredi Karti";
```

```
    // Taksit arttikca komisyon artar
    public override decimal KomisyonHesapla() => Tutar * 0.015m * TaksitSayisi;
}

public class HavaleOdemesi : Odeme
{
    public HavaleOdemesi(decimal tutar) : base(tutar) { }

    public override string YontemAdi => "Havale/EFT";
    public override decimal KomisyonHesapla() => 0m;      // komisyonsuz
}

public class KapidaOdeme : Odeme
{
    public KapidaOdeme(decimal tutar) : base(tutar) { }

    public override string YontemAdi => "Kapida Odeme";
    public override decimal KomisyonHesapla() => 25m;     // sabit hizmet bedeli
}
```

Kod 6.2 — Polymorphism ile tek noktadan işleme

```
List<Odeme> odemeler =
[
    new KrediKartiOdemesi(1500m, 6),
    new HavaleOdemesi(2400m),
    new KapidaOdeme(320m)
];

// ===== POLYMORPHISM: tek dongü, uc farkli hesaplama =====
foreach (Odeme o in odemeler)
    Console.WriteLine(o);              // her nesne kendi ToString + komisyonu

Console.WriteLine($"\nGenel toplam: {odemeler.Sum(o => o.ToplamTutar()):C}");
```

Kod 6.2 çıktısı (örnek)

```
[3F9A2C71] Kredi Karti   1.500,00 TL +   135,00 TL =  1.635,00 TL
[B8E01D4A] Havale/EFT    2.400,00 TL +     0,00 TL =  2.400,00 TL
[7C25F930] Kapida Odeme    320,00 TL +    25,00 TL =    345,00 TL

Genel toplam: 4.380,00 TL
```

> **[+] Yeni bir ödeme yöntemi eklemek**
>
> `KriptoOdemesi : Odeme` sınıfını yazıp iki üyeyi doldurmanız yeterlidir. **Kod 6.2'deki döngü hiç değişmez.** İşte dört prensibin birlikte çalıştığında sağladığı fayda budur.

6.1 Sık yapılan hatalar

Hata	Neden yanlış?	Doğrusu
Her şeyi `public` yapmak	Sınıf kendi kurallarını koruyamaz	Field `private`, erişim property üzerinden
Sadece kod paylaşmak için kalıtım kurmak	Yanlış **is-a** ilişkisi, kırılgan hiyerarşi	Composition kullanın veya ortak kodu ayrı bir sınıfa alın
`virtual` yazmayı unutmak	Alt sınıf `override` edemez, örtük `new` olur	Ezilecek metodu `virtual` veya `abstract` işaretleyin
Tip kontrolü zinciri (`if is`…)	Her yeni tipte mevcut kodu değiştirmek gerekir	Davranışı sanal metoda taşıyın
`Equals` ezip `GetHashCode`'u ezmemek	Koleksiyonlarda beklenmedik davranış	İkisi **her zaman** birlikte ezilir
Çok derin kalıtım hiyerarşisi	Anlaşılması ve değiştirilmesi zorlaşır	2–3 seviyeyi geçmeyin

Haftanın Özeti

1. **Encapsulation**: Field'ları `private` yapın, erişimi property üzerinden verin, doğrulamayı sınıfın içine gömün. Nesne hiçbir zaman geçersiz duruma düşmemelidir.
2. **Inheritance**: `:` ile kurulur, `base(...)` ile temel constructor çağrılır, `protected` üyeler alt sınıflara açıktır. Yalnızca gerçek **is-a** ilişkisinde kullanın.
3. **Polymorphism — overloading**: Aynı sınıfta, farklı imza, derleme zamanında çözülür.
4. **Polymorphism — overriding**: `virtual` + `override` ikilisi, çalışma zamanında nesnenin gerçek tipine göre çözülür. `new` ise ezmez, gizler.
5. **Abstraction**: `abstract` sınıf örneklenemez; `abstract` üyeler alt sınıflar tarafından doldurulmak zorundadır. Ortak kod paylaşımı gerekiyorsa abstract class, yalnızca yetenek sözleşmesi gerekiyorsa interface seçin.
6. Dördü birlikte, **yeni tip eklerken mevcut kodu değiştirmeyi gerektirmeyen** bir tasarım üretir.

Anahtar Terimler Sözlüğü

Terim (EN)	Türkçe	Kısa tanım
encapsulation	kapsülleme	Veriyi gizleyip kontrollü erişim sağlama
invariant	değişmez kural	Nesnenin her an sağlaması gereken koşul
inheritance	kalıtım	Bir sınıfın başka bir sınıfın üyelerini devralması
base class	temel sınıf	Üyelerini devreden üst sınıf
derived class	türetilmiş sınıf	Devralan alt sınıf
is-a / has-a	-dır / -sahiptir	Kalıtım ile bileşim arasındaki seçim ölçütü
polymorphism	çok biçimlilik	Aynı çağrının tipe göre farklı davranması
overloading	aşırı yükleme	Aynı adlı metodun farklı imzalarla tanımlanması
overriding	ezme	Türetilmiş sınıfın temel metodu yeniden yazması
virtual	sanal	Ezilebilir olduğunu belirten anahtar kelime
method hiding	metot gizleme	`new` ile temel metodu perdeleme
upcasting	yukarı dönüşüm	Alt tipi üst tip olarak tutma (güvenli)
downcasting	aşağı dönüşüm	Üst tipi alt tipe çevirme (riskli)
abstraction	soyutlama	Ne yapıldığını gösterip nasılını gizleme
abstract class	soyut sınıf	Örneklenemeyen, sözleşme tanımlayan sınıf
sealed	mühürlü	Kalıtımı veya ezmeyi kapatan anahtar kelime

Alıştırmalar

1. **Encapsulation:** `Sicaklik` sınıfı yazın. `Celsius` property'si -273,15'in altına inemesin. `Fahrenheit` ve `Kelvin` hesaplanan property olsun.
2. **Inheritance:** `Arac` temel sınıfından `Otomobil`, `Motosiklet` ve `Kamyon` türetin. Ortak üyeler: `Marka`, `Model`, `Yil`. Her birinde `VergiHesapla()` farklı çalışsın.

3. **Polymorphism:** Yukarıdaki araçları bir `List<Arac>` içinde tutup toplam vergiyi tek bir `foreach` ile hesaplayın. Hiç `if` kullanmayın.
4. **Abstraction:** `Calisan` soyut sınıfını yazın. `MaasHesapla()` soyut, `BilgiYazdir()` sanal olsun. `SaatlikCalisan` ve `MaasliCalisan` sınıflarıyla doldurun.
5. `override` ile `new` arasındaki farkı gösteren kısa bir program yazın. Aynı nesneyi hem temel hem türetilmiş tip değişkeninde tutup çıktıları karşılaştırın.
6. `Kitap` sınıfında `ToString()`, `Equals()` ve `GetHashCode()` metotlarını ISBN'e göre ezin. İki farklı nesnenin aynı ISBN ile eşit sayıldığını gösterin.
7. **Mini proje:** Bir kütüphane otomasyonu tasarlayın. `Materyal` soyut sınıfından `Kitap`, `Dergi`, `DVD` türetin. Her materyalin farklı bir `OduncSuresi` ve `GecikmeCezasiHesapla(int gun)` davranışı olsun. Dört prensibi de kullandığınız yerleri kod içinde yorum satırlarıyla işaretleyin.

Ekstra Okuma ve Kaynaklar

- **Microsoft Learn — Nesne Yönelimli Programlama Eğitimi (Türkçe)**
  Dört prensibi adım adım kod yazarak öğreten resmî interaktif eğitim.
  https://learn.microsoft.com/tr-tr/dotnet/csharp/fundamentals/tutorials/oop
- **Inheritance — C# Belgeleri**
  Kalıtım, soyut sınıflar ve sanal metotların derinlemesine anlatımı; banka hesabı örneği.
  https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/tutorials/inheritance
- **Polymorphism — C# Programming Guide**
  virtual/override/new arasındaki farkları bellek düzeyinde açıklayan resmî sayfa.
  https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/polymorphism
- **abstract — C# Dil Referansı**
  Soyut sınıf ve üyelerin kuralları, sık yapılan derleme hataları ile birlikte.
  https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/abstract
- **Abstract classes vs interfaces — .NET Tasarım Rehberi**
  Microsoft'un kendi kütüphanelerini tasarlarken kullandığı karar ölçütleri.
  https://learn.microsoft.com/en-us/dotnet/standard/design-guidelines/choosing-between-class-and-interface
- **Object.Equals ve GetHashCode'u doğru ezmek**
  Değer eşitliği tanımlarken uyulması gereken sözleşme ve tam örnek.
  https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/statements-expressions-operators/how-to-define-value-equality-for-a-type
- **C# 14 — field anahtar kelimesi**
  Kod 2.4'te kullandığımız yeni yazımın resmî belgesi ve kısıtları.
  https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/field
- **SOLID prensipleri — Giriş**
  Bu haftaki prensiplerin üzerine kurulu, ileri seviye tasarım ilkeleri.
  https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/architectural-principles
- **Refactoring Guru — Design Patterns (C# örnekleriyle)**
  Dört prensibin gerçek projelerde nasıl kalıplaştığını gösteren görsel kaynak.
  https://refactoring.guru/design-patterns/csharp